CSA04-Operating Systems

TASK 5 – Examine System Calls

Name: V. Reddy Sreya **Regno: 192525124**

1. Aim

To examine how a user program requests services from the operating system using system calls.

2. Categories of System Calls

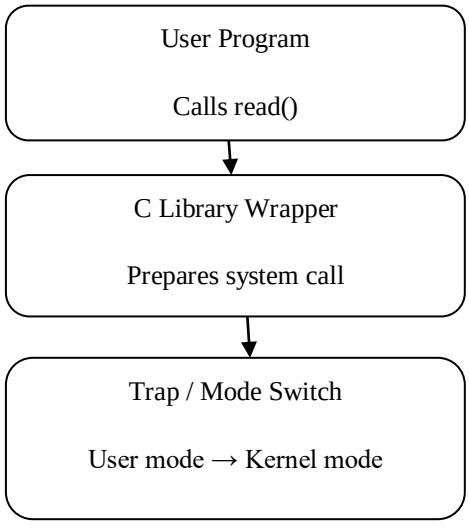
Category	Purpose	Examples
Process control	Creates and manages processes	fork(), exec(), exit()
File management	Creates, reads, writes and closes files	open(), read(), write(), close()
Device management	Accesses and controls devices	ioctl(), device read/write
Information maintenance	Gets system and process information	getpid(), time()
Communication	Exchanges data between processes	pipe(), socket()
Protection	Controls access and permissions	chmod(), chown()

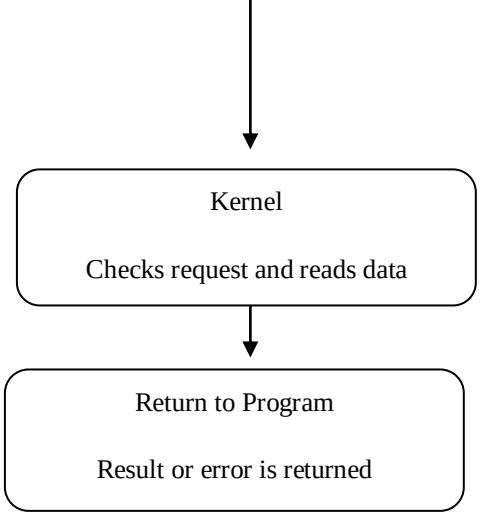
3. Steps Involved in the read() System Call

When a program reads data from a file, the following steps occur:

1. Library call: The program calls the read() function.
2. Trap: The library wrapper requests the operating system service.
3. Mode switch: The CPU changes from user mode to kernel mode.
4. Kernel work: The kernel checks the file descriptor, permissions and requested data, then reads the data.
5. Return: The kernel returns the number of bytes read or an error. The CPU switches back to user mode.

Flow of a system call:





4. Passing Parameters to the Kernel

System-call parameters can be passed in three ways:

- Registers: Parameters are placed directly in CPU registers.
- Memory block: Parameters are stored in a memory block, and its address is passed.
- Stack: Parameters are pushed onto the program stack.

The method depends on the processor architecture and operating system.

5. How strace Shows System Calls

strace is a Linux command-line tool used to observe the system calls made by a program. It helps us understand how a program communicates with the kernel.

Sample C program using fopen() and fread()

Save the following as readfile.c:

```
#include <stdio.h>

int main()

{

  FILE *fp;

   char buffer[100];

   fp = fopen("sample.txt", "r");

   if (fp == NULL)

   {

      printf("File cannot be opened\n");
```

```
        return 1;
    }
    fread(buffer, 1, sizeof(buffer) - 1, fp);
    buffer[sizeof(buffer) - 1] = '\0';
    printf("File content: %s\n", buffer);
    fclose(fp);
    return 0;
}
```

Steps to execute in Ubuntu:

1. Create a file named `sample.txt` and enter some text.
2. Save the C program as `readfile.c`.
3. Open the terminal in that folder.
4. Compile and run:

   gcc readfile.c -o readfile

   ./readfile

5. Trace the system calls:

   strace -e trace=openat,read,write,close ./readfile

Typical evidence to look for:

openat(..., "sample.txt", O_RDONLY) = 3

read(3, "...", 100) = ...

write(1, "File content: ...", ...) = ...

close(3) = 0

The exact output may differ depending on the Linux system and file contents. openat() opens the file, read() obtains data, write() displays output, and close() closes the file.

6. Alternative Concept and Complexity Analysis

Alternative: C library functions versus direct system calls

C library functions such as fopen(), fread() and fwrite() provide convenient file handling. They use operating-system system calls internally.

Feature	C library functions	Direct system calls
Opening a file	fopen()	open()
Reading data	fread()	read()
Writing data	fwrite()	write()
Buffering	Usually provides buffering	Application manages its buffer
System-call overhead	Reduced through buffering	Depends on requested buffer size

System-call count for a file of n bytes:

- With a library buffer of 4096 bytes, approximately \(n/4096\) read or write system calls are needed.
- With direct system calls using a buffer of b bytes, approximately \(n/b\) calls are needed.
- Opening and closing the file add a small, fixed number of system calls.

For example, for a 40,960-byte file:

Method	Buffer size	Approximate read calls
Library buffering	4096 bytes	10
Direct system calls	1024 bytes	40

Extra buffer space: The C library uses additional memory to store buffered data. A 4096-byte buffer requires approximately 4 KB per stream, depending on the implementation. Direct system calls can use an application-controlled buffer.

Evaluation: Library buffering is generally more efficient for repeated small file operations because it reduces system-call overhead. Direct system calls provide more control over data transfer but may require more calls when small buffers are used.

7. Result

Thus, system calls provide the connection between user programs and the operating system kernel. The strace command helps identify system calls, while C library buffering reduces the number of system calls and improves file-handling efficiency.